

LECTURE 5

Security and Protection: An Introduction

CSC323 · Operating Systems

COMSATS University Islamabad, Lahore Campus

In This Lecture

- The distinction between protection and security — two related but genuinely different concerns
- The principle of least privilege, and why it organizes almost every protection mechanism
- The general idea of an access matrix: domains (subjects) against objects, with rights in between
- The basics of authentication — something you know, have, or are
- Why protection matters even on a machine with exactly one user

Before the Mechanisms, the Question

Every mechanism this course covers later — every process, every open file, every shared segment of memory — is, from the OS's point of view, a resource that something or someone might misuse, on purpose or by accident.

- This lecture asks: who is allowed to do what to this system, and how does the OS enforce the answer?
- It is not a deep dive into cryptography or access-control implementations — that belongs to a dedicated security course
- It is the vocabulary and mental model needed before processes, files, and shared memory start needing protection

Protection vs. Security

These two words get used almost interchangeably in casual conversation, but an operating systems course needs to keep them separate — they answer different questions.

Protection

"These two trusted processes are both allowed on this machine — how do I stop one from corrupting the other's files by mistake?"

Security

"Someone outside this system is actively trying to get in, steal data, or run code they were never given permission to run — how do I stop them?"

Protection mechanisms are security's building blocks. A system with perfect protection is much harder to compromise — even a successfully-launched attack is stuck inside the narrow permissions the protection system grants it.

Protection or Security? A Quick Test

Ask who the actors are.

- If every actor involved is already running legitimately on the machine, and the question is “should this process/user be allowed to touch that resource” — it's protection
- If the question involves someone or something that doesn't belong on the system trying to get in or do damage anyway — it's security
- Weak protection turns even a minor security breach into a total one — this course introduces protection first because it underlies security

Why Protection Matters Even on a Single-User System

It's tempting to think protection only matters on a shared, multi-user machine. But protection matters even on a laptop with one human user and one account, for a simple reason: a buggy program is a threat too.

- A pointer bug that writes past a buffer, a misconfigured script that deletes the wrong directory, a runaway process that consumes all memory — all need containing
- The OS can't tell “a hostile process overwriting memory” from “a buggy process doing the same thing by accident” — and it shouldn't need to
- Sound protection stops both with the same mechanism: never letting either one touch memory it wasn't assigned

Trust Is Not the Same as Correctness

A program can be fully trusted and still not be correct. Even code written in good faith by a competent programmer, running under your own account, is not correct by virtue of being trusted. Protection's job is to limit the damage a process can do if it misbehaves — whether that misbehavior comes from an attacker, a bug, or a careless typo. The resource being protected doesn't care which.

The Principle of Least Privilege

Every process, user, and program should operate with the smallest set of access rights necessary to accomplish its task, and no more.

- A text editor needs to read/write the file you opened — not every other user's private files
- A print spooler needs the printer queue — not your email
- Least privilege isn't about distrust — it's a default posture: grant the minimum, widen only when a justified need appears
- The benefit compounds exactly in the buggy-program scenario: damage is bounded by the narrow permission set actually held

Least Privilege, Applied to a Simple Example

Without least privilege

A spell-checker process is given read/write access to the whole filesystem “to be safe”



With least privilege

The same process can only read/write the one document it was asked to check — a bug in it can't touch anything else

Access Control: The Idea of an Access Matrix

If least privilege says what should happen, the OS still needs a concrete way to represent who is allowed to do what. The classic conceptual model is the access matrix.

- Rows are domains (subjects) — a user, or more precisely a process running on that user's behalf
- Columns are objects — the resources worth protecting: files, devices, memory segments, even other processes
- Each cell lists the access rights that domain holds over that object — read, write, execute, delete, and so on

An Access Matrix — Domains Against Objects

Domain	report.docx	/etc/passwd-equivalent	Printer	payroll.db
Student user	read, write	—	print	—
Instructor user	read	—	print	—
System administrator	read, write, delete	read, write	print	—
Payroll process	—	—	—	read, write

A right is specific to one (domain, object) pair. The payroll process can read/write payroll.db while no ordinary user can touch it at all; the student can write report.docx but has no rights over the printer configuration or the system file. Every cell is its own independent decision.

Implementing the Access Matrix

This is the idea, not a specific implementation — real systems rarely store a literal, giant, mostly-empty grid (the matrix is usually enormous and overwhelmingly sparse).

- Column-by-column: an access control list (ACL) attached to each object, naming which domains may do what to it
- Row-by-row: a capability list attached to each domain, naming which objects it may touch and how
- Either way, the OS's job is the same: look up some domain, requesting some operation, on some object — and allow or deny it

Authentication Basics

Access control only works if the system first knows, with reasonable confidence, which domain is making a request. That identification step is authentication, built from one or more of three categories of evidence:

Something you know

A password, a PIN, a security question. Cheap, but only as strong as the secret's unpredictability — can be guessed, stolen, or shared.

Something you have

A physical token, a smart card, a one-time code to your phone. Harder to steal remotely, but can be physically lost or stolen.

Something you are

A biometric: fingerprint, face, retina. Hard to lose or forget, but can't be changed once compromised.

Multi-Factor Authentication Is Just Combining Categories

MFA sounds more complex than it is. It simply means requiring evidence from more than one of the three categories before granting access — a password (something you know) plus a code sent to your phone (something you have). Combining categories matters more than piling up evidence from the same category, because an attacker who defeats one category still has to separately defeat an entirely different one.

Authentication and Access Control, Together

Authentication answers “who are you?” Access control (the matrix) then answers “given who you are, what are you allowed to do?”

- The two are sequential and both necessary
- A perfect access matrix is worthless if the system can be tricked about whose row to look up
- A perfect authentication scheme is worthless if, once logged in, every domain is simply granted access to everything

Key Takeaways

- Protection controls access among entities already trusted on the system; security defends against threats from outside that circle of trust
- Protection matters even on a single-user machine — a buggy, non-malicious program is just as capable of damaging a resource as a malicious one
- The principle of least privilege — grant the smallest set of rights needed, and nothing more — organizes almost every protection mechanism
- An access matrix (domains/subjects $\times$ objects, with rights in each cell) is the conceptual model behind access control; real systems use ACLs or capability lists
- Authentication establishes who is making a request (something you know, have, or are); access control then decides what that identity is allowed to do